

Chapter 32: Dynamic Programming

Personal Notes

If recursion is the gateway to DSA, dynamic programming (DP) is the masterclass. It's the technique that turns exponential algorithms into polynomial ones — taking problems that look impossible (like "find the longest common subsequence of two strings") and solving them in $O(n \times m)$.

DP intimidates a lot of programmers, but the core idea is dead simple: if you're solving the same subproblem over and over, REMEMBER the answer. That's it. The rest is bookkeeping and pattern recognition — and once you've solved 10 DP problems, the patterns start repeating.

1. What is Dynamic Programming?

Dynamic Programming is a technique for solving problems by breaking them into OVERLAPPING SUBPROBLEMS and storing the results so you never solve the same subproblem twice.

Two Properties a Problem Must Have

- Optimal Substructure — the optimal answer for the whole problem can be built from optimal answers to smaller subproblems
- Overlapping Subproblems — naive recursion would solve the same subproblem MANY TIMES

If both are true, DP applies. If only the first is true (no overlap), you usually have divide-and-conquer instead, like merge sort.

DP in one sentence: Recursion + remember the answers. Or, equivalently, build up answers from the smallest cases to the biggest. Either way, every subproblem is solved EXACTLY ONCE.

2. The Fibonacci Story (Revisited)

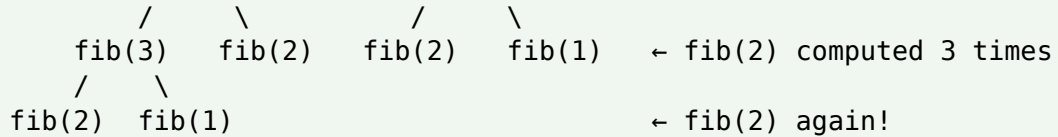
You saw this in Chapter 29. It's worth revisiting because it perfectly demonstrates WHY DP exists.

Naive Recursive — $O(2^n)$

```
public static int fib(int n) {
    if (n <= 1) return n;
    return fib(n - 1) + fib(n - 2);
}
```

The Problem

fib(5)
 / \
 fib(4) fib(3) ← fib(3) computed twice



For fib(50): about 2.5 BILLION calls.
Each subproblem is solved many times.

The Fix — Memoization

```

static int[] memo = new int[100];

public static int fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != 0) return memo[n];          // already solved
    return memo[n] = fib(n - 1) + fib(n - 2);
}
  
```

Same recursion structure, plus a memo array. Each fib(k) is now computed ONCE. Time goes from $O(2^n)$ to $O(n)$. fib(50) is instant.

3. The Two Approaches: Memoization vs Tabulation

Every DP problem can be solved in two styles. Both produce the same answer; they just walk the same calculations in different directions.

3.1 Memoization (Top-Down)

Start with the original problem. Recurse normally, but cache results before returning. "Solve from the top, store on the way back."

```

static Integer[] memo;

public static int fib(int n) {
    if (n <= 1) return n;
    if (memo[n] != null) return memo[n];
    return memo[n] = fib(n - 1) + fib(n - 2);
}
  
```

3.2 Tabulation (Bottom-Up)

Build a table from the smallest subproblem upward. No recursion — just a loop. "Compute fib(2), then fib(3), then fib(4), ... using the values already in the table."

```

public static int fib(int n) {
    if (n <= 1) return n;
    int[] dp = new int[n + 1];
    dp[0] = 0;
    dp[1] = 1;
  
```

```

    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}

```

3.3 Which to Use?

Aspect	Memoization	Tabulation
Direction	Top → Down (start from the goal)	Bottom → Up (start from base cases)
Uses	Recursion + cache	Loop + table
Memory	Uses stack space (can overflow)	No recursion — safer for big n
Speed	Slight function-call overhead	Usually a bit faster (loops)
When easier	When the recursion is natural	When you can list subproblems in order
Risk	StackOverflowError for very deep DP	Off-by-one errors in the table

Interview tip: Most interviewers want you to START with memoization (it follows naturally from the recursive solution), then OPTIMIZE to tabulation if asked. This shows you understand both the problem structure AND how to refine. Saying "first I'll write recursion, then add a memo, then convert to bottom-up" is the gold standard.

4. The 5-Step Recipe for Any DP Problem

Every DP problem can be cracked with the same questions. Memorize this sequence:

1. Define the STATE — what variable(s) uniquely identify a subproblem? (e.g. "index i" or "i, j" or "index, capacity")
2. Define the RECURRENCE — how does the answer for state S depend on smaller states?
3. Identify the BASE CASE — the smallest subproblems with known answers
4. Decide the ORDER — for memoization just recurse; for tabulation fill the table from smallest to largest
5. Extract the ANSWER — which entry in the table or recursion call gives the final result?

Get these five right and the code writes itself. Almost every DP problem in an interview maps onto this template — the differences are just in how you define state and recurrence.

5. Classic Problem 1 — Climbing Stairs

You're climbing a staircase with N steps. You can climb 1 or 2 steps at a time. How many distinct ways are there to reach the top?

Examples

- $N = 2 \rightarrow 2$ ways: (1,1) or (2)
- $N = 3 \rightarrow 3$ ways: (1,1,1), (1,2), (2,1)
- $N = 4 \rightarrow 5$ ways
- $N = 5 \rightarrow 8$ ways

Recognize the sequence? It's Fibonacci.

Apply the 5-Step Recipe

- State — $dp[i]$ = number of ways to reach step i
- Recurrence — $dp[i] = dp[i-1] + dp[i-2]$ (you came from either 1 step below or 2 below)
- Base case — $dp[0] = 1, dp[1] = 1$
- Order — fill from $i = 2$ upward
- Answer — $dp[N]$

Tabulation Code

```
public static int climbStairs(int n) {
    if (n <= 1) return 1;
    int[] dp = new int[n + 1];
    dp[0] = 1;
    dp[1] = 1;
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2];
    }
    return dp[n];
}
```

Space Optimization

Notice you only need the LAST TWO values, not the whole table. Drop the array:

```
public static int climbStairs(int n) {
    if (n <= 1) return 1;
    int prev1 = 1, prev2 = 1;
    for (int i = 2; i <= n; i++) {
        int curr = prev1 + prev2;
        prev2 = prev1;
        prev1 = curr;
    }
    return prev1;
}
```

O(n) time, O(1) space. Many DP problems have this kind of optimization once you see the pattern.

6. Classic Problem 2 — House Robber

A street has N houses in a row, each with some money. You can't rob two ADJACENT houses (alarm system). What's the maximum amount you can rob?

Apply the Recipe

- State — $dp[i]$ = max money robbed considering houses 0..i
- Recurrence — at each house, you either ROB it (skip i-1, add to $dp[i-2]$) or SKIP it (carry $dp[i-1]$)
- $dp[i] = \max(dp[i-1], dp[i-2] + \text{nums}[i])$
- Base cases — $dp[0] = \text{nums}[0]$, $dp[1] = \max(\text{nums}[0], \text{nums}[1])$
- Answer — $dp[n-1]$

Code

```
public static int rob(int[] nums) {
    if (nums.length == 0) return 0;
    if (nums.length == 1) return nums[0];

    int[] dp = new int[nums.length];
    dp[0] = nums[0];
    dp[1] = Math.max(nums[0], nums[1]);

    for (int i = 2; i < nums.length; i++) {
        dp[i] = Math.max(dp[i - 1], dp[i - 2] + nums[i]);
    }

    return dp[nums.length - 1];
}
```

Dry Run for [2, 7, 9, 3, 1]

```
i=0: dp[0] = 2
i=1: dp[1] = max(2, 7) = 7
i=2: dp[2] = max(7, 2 + 9) = 11
i=3: dp[3] = max(11, 7 + 3) = 11
i=4: dp[4] = max(11, 11 + 1) = 12
```

Answer: 12 (rob houses 0, 2, 4: 2 + 9 + 1 = 12)

7. Classic Problem 3 — Coin Change

Given coin denominations and a target amount, find the MINIMUM number of coins needed. Each coin can be used unlimited times. Return -1 if impossible.

Example: coins = [1, 2, 5], amount = 11 → answer 3 (5 + 5 + 1)

Apply the Recipe

- State — $dp[i]$ = min coins needed to make amount i
- Recurrence — for each coin c , $dp[i] = \min(dp[i], dp[i-c] + 1)$
- Base case — $dp[0] = 0$ (zero coins for amount 0)
- Order — fill from amount = 1 to target
- Answer — $dp[amount]$, or -1 if unreachable

Code

```
public static int coinChange(int[] coins, int amount) {
    int[] dp = new int[amount + 1];
    Arrays.fill(dp, amount + 1);    // "infinity" marker
    dp[0] = 0;

    for (int i = 1; i <= amount; i++) {
        for (int coin : coins) {
            if (coin <= i) {
                dp[i] = Math.min(dp[i], dp[i - coin] + 1);
            }
        }
    }

    return dp[amount] > amount ? -1 : dp[amount];
}
```

Why amount+1 as infinity: You can never use more than (amount) coins (each coin is at least 1). So amount+1 is safely larger than any real answer — and easy to spot as "unreachable" at the end.

8. Classic Problem 4 — Longest Increasing Subsequence (LIS)

Given an array, find the length of the longest strictly increasing subsequence. The elements don't need to be contiguous, just in order.

Example: [10, 9, 2, 5, 3, 7, 101, 18] → LIS is [2, 3, 7, 101] or [2, 5, 7, 18] → length 4

Apply the Recipe

- State — $dp[i]$ = length of LIS ENDING at index i
- Recurrence — $dp[i] = 1 + \max(dp[j])$ for all $j < i$ where $nums[j] < nums[i]$
- Base case — every $dp[i]$ starts as 1 (a single element is its own LIS of length 1)
- Order — fill from $i = 0$ upward, inner loop over $j < i$
- Answer — the MAX value in the dp array (LIS can end at any index)

Code

```
public static int lengthOfLIS(int[] nums) {
    int n = nums.length;
    int[] dp = new int[n];
    Arrays.fill(dp, 1);

    int best = 1;
    for (int i = 1; i < n; i++) {
        for (int j = 0; j < i; j++) {
            if (nums[j] < nums[i]) {
                dp[i] = Math.max(dp[i], dp[j] + 1);
            }
        }
        best = Math.max(best, dp[i]);
    }

    return best;
}
```

$O(n^2)$ time, $O(n)$ space. There's also an $O(n \log n)$ solution using binary search — worth learning for harder interviews.

9. Classic Problem 5 — 0/1 Knapsack

You have N items, each with weight and value. Knapsack holds at most W weight. Each item can be taken 0 or 1 times ("0/1" knapsack). Maximize total value.

Example: weights = [1, 3, 4, 5], values = [1, 4, 5, 7], $W = 7 \rightarrow$ optimal value = 9 (items 1 and 3)

Apply the Recipe — Now with 2D state

- State — $dp[i][w]$ = max value using first i items with capacity w
- Recurrence — for each (i, w) :
 - INCLUDE item i (if it fits): $dp[i-1][w - wt[i]] + val[i]$
 - EXCLUDE item i : $dp[i-1][w]$
- $dp[i][w]$ = max of the two
- Base — $dp[0][w] = 0$ (no items chosen yet)
- Answer — $dp[n][W]$

Code

```
public static int knapsack(int[] weights, int[] values, int W) {
    int n = weights.length;
    int[][] dp = new int[n + 1][W + 1];

    for (int i = 1; i <= n; i++) {
        for (int w = 0; w <= W; w++) {
```

```

        // Default: don't take this item
        dp[i][w] = dp[i - 1][w];

        // If item fits, consider taking it
        if (weights[i - 1] <= w) {
            int take = dp[i - 1][w - weights[i - 1]] + values[i - 1];
            dp[i][w] = Math.max(dp[i][w], take);
        }
    }

    return dp[n][W];
}

```

2D DP is the next frontier: Many advanced DP problems use 2D state — knapsack, edit distance, longest common subsequence, matrix path problems. The pattern is the same: fill the table from base cases outward, using values already computed.

10. The Memoization → Tabulation Conversion

Once you have a memoized solution, converting it to tabulation is mechanical. Here's the climb-stairs example shown both ways:

Memoization (Top-Down)

```

static Integer[] memo;

public static int climb(int n) {
    if (n <= 1) return 1;
    if (memo[n] != null) return memo[n];
    return memo[n] = climb(n - 1) + climb(n - 2);
}

```

Tabulation (Bottom-Up) — Mechanical Conversion

```

public static int climb(int n) {
    int[] dp = new int[n + 1];
    dp[0] = 1; dp[1] = 1;           // base cases
    for (int i = 2; i <= n; i++) {
        dp[i] = dp[i - 1] + dp[i - 2]; // same recurrence
    }
    return dp[n];
}

```

Both compute the EXACT SAME values. The memoized version starts at n and walks down to 0. The tabulated version starts at 0 and walks up to n . Different direction, same answer.

Conversion Steps

- Find the state shape (e.g. $dp[n]$ or $dp[i][w]$)

7. Allocate the table
8. Fill in base cases at the start
9. Loop in the order that lets each cell read from already-filled cells
10. Use the same recurrence formula
11. Return the answer cell (often $dp[n]$ or $dp[n-1][m-1]$)

11. Recognizing a DP Problem

Common signals that scream "DP!"

- "Find the maximum / minimum / longest / shortest..." with overlapping options
- "How many ways to ..." — counting problems
- Recursive structure where you're SOLVING THE SAME SUBPROBLEM repeatedly
- "Yes/no" — can you reach a target / form a string / partition a set?
- Problems involving choices — "take or skip," "go left or right," "split here or there"
- Sequences and strings — LIS, LCS, edit distance, palindrome substrings
- Grid problems — paths from corner to corner, minimum cost paths

Common confusion: DP vs Backtracking Backtracking explores ALL possibilities (and you USE them all). DP applies when subproblems REPEAT and you only need the optimal one. If the same partial state appears multiple times in your recursion tree, DP wins. If every state is unique, backtracking is the right tool.

12. Time & Space Complexity

DP complexity is usually:

- Time = number of unique states $\times$ work per state
- Space = size of the DP table

Problem	Time	Space
Climbing Stairs	$O(n)$	$O(n)$ or $O(1)$ with rolling vars
House Robber	$O(n)$	$O(n)$ or $O(1)$ with rolling vars
Coin Change	$O(n \times W)$	$O(W)$
LIS (basic)	$O(n^2)$	$O(n)$
LIS (binary search)	$O(n \log n)$	$O(n)$
0/1 Knapsack	$O(n \times W)$	$O(n \times W)$ or $O(W)$ with 1D table
Edit Distance	$O(m \times n)$	$O(m \times n)$

Problem	Time	Space
LCS	$O(m \times n)$	$O(m \times n)$

13. Common Mistakes

Mistake 1: Wrong state definition

If your state doesn't UNIQUELY identify a subproblem, you'll get wrong answers. For LIS, $dp[i]$ must mean "LIS ending at i ," not "LIS in $nums[0..i]$." A subtle but crucial distinction.

Mistake 2: Off-by-one in the table

Especially in knapsack-style problems with 1-indexed items vs 0-indexed arrays. Always check: does $dp[i]$ mean "first i items" or "index i "? Pick one and stick with it.

Mistake 3: Wrong base case

```
// House Robber – wrong base
dp[1] = nums[1];           // x ignores nums[0]

// Correct
dp[1] = Math.max(nums[0], nums[1]); // ✓
```

Mistake 4: Filling table in wrong order

In 2D DP, you must fill cells in an order such that every recurrence reads cells that are ALREADY filled. Get the loop direction wrong and you'll read garbage values.

Mistake 5: Returning the wrong cell

LIS doesn't return $dp[n-1]$ — it returns $\max(dp)$. House Robber DOES return $dp[n-1]$. Always think about WHICH state holds the final answer.

Mistake 6: Recomputing inside recursion

```
// Looks like memoization but ISN'T:
int solve(int i) {
    if (memo[i] != 0) return memo[i];
    int answer = solve(i - 1) + solve(i - 2);
    return answer;           // x forgot to store!
}

// Fix:
int solve(int i) {
    if (memo[i] != 0) return memo[i];
    return memo[i] = solve(i - 1) + solve(i - 2); // ✓
}
```

Mistake 7: Using 0 as the sentinel value

If 0 is a valid answer, you can't use 0 to mean "not yet computed." Use -1, null (with Integer[]), or a separate boolean[] to track computed cells.

14. Best Practices

- Start by writing a clean recursive solution — confirm correctness first
- Add memoization to that recursion — usually just adds 2 lines
- Convert to tabulation if the interviewer asks for it, or if recursion depth is a worry
- Apply space optimization (rolling variables, 1D tables) only after the basic version works
- Always state your dp[i] definition in plain English before coding
- Dry-run with a tiny input (n = 4 or 5) before submitting
- For 2D DP, draw the table and fill it by hand for a small example
- Use Integer[] (not int[]) for memo arrays when 0 is a valid answer

15. Quick Reference

Concept	Meaning
DP	Dynamic Programming — solve once, remember the answer
State	Variable(s) that uniquely identify a subproblem
Recurrence	Formula expressing dp[state] in terms of smaller states
Base case	Subproblems with known answers, no recursion needed
Memoization	Top-down DP using recursion + cache
Tabulation	Bottom-up DP using loops + table
Optimal substructure	Optimal whole = optimal pieces combined
Overlapping subproblems	The same subproblem comes up many times
1D DP	State has one variable: dp[i]
2D DP	State has two variables: dp[i][j]
Space optimization	Reduce memory when only last few states are needed

16. Practice Problems

Solve in order — each one builds on the patterns of the previous.

1D DP — Warm-ups

- 12. Fibonacci — both memoized and tabulated versions.
- 13. Climbing Stairs — $N=2$ stairs, two-step option.
- 14. Climbing Stairs with VARIABLE step sizes (1, 2, 3 allowed).
- 15. Min Cost Climbing Stairs — cost on each step, choose 1 or 2 steps.
- 16. House Robber — classic.
- 17. House Robber II — houses in a CIRCLE (first and last are adjacent).

1D DP — Choice Problems

- 18. Coin Change — minimum coins to reach amount.
- 19. Coin Change II — number of ways to make amount.
- 20. Word Break — can a string be split into dictionary words?
- 21. Decode Ways — count ways to decode a digit string into letters (1=A, 26=Z).

2D DP — Grid Problems

- 22. Unique Paths — count paths from top-left to bottom-right (right and down only).
- 23. Unique Paths II — same, but some cells are blocked.
- 24. Minimum Path Sum — find the path with minimum sum from top-left to bottom-right.

Sequence DP

- 25. Longest Increasing Subsequence (basic $O(n^2)$).
- 26. Longest Common Subsequence — given two strings.
- 27. Edit Distance — min insert/delete/replace to convert one string to another.
- 28. Longest Palindromic Subsequence.
- 29. Longest Palindromic Substring.

Knapsack Family

- 30. 0/1 Knapsack — items used at most once.
- 31. Unbounded Knapsack — items can be used unlimited times.
- 32. Partition Equal Subset Sum — can the array be split into two equal-sum halves?
- 33. Target Sum — count ways to assign + and - to numbers so the sum equals target.

17. Final Takeaway

DP is intimidating until you realize it's just "recursion + cache." Once you've solved 10-15 problems, the patterns repeat — climbing stairs, knapsack, LIS, edit distance — and new problems start looking like variations of ones you've already solved.

The real skill is RECOGNIZING the pattern. Most interview DP problems are disguised versions of these classics: "max sum subsequence," "longest something," "number of ways," "min cost path." Once you know the canonical forms, transformation becomes the easy part.

Key Takeaway: DP = recursion + memo, or equivalently, build the answer bottom-up in a table. Master the 5-step recipe: STATE → RECURRENCE → BASE → ORDER → ANSWER. Start with memoization, optimize to tabulation, then space-optimize. Practice the canonical problems (Fibonacci, climbing stairs, house robber, coin change, LIS, knapsack, LCS, edit distance) — they cover 90% of DP interviews.

18. What's Next?

With DP under your belt, the remaining DSA pillars are:

- Greedy Algorithms — when local optima lead to global optimum
- Graphs — DFS, BFS, shortest path (Dijkstra), topological sort
- Heaps & Priority Queues — top-K problems, Dijkstra's helper
- Tries — prefix trees for word search, autocomplete
- Union-Find (DSU) — connected components, cycle detection
- Bit Manipulation — fast tricks with binary representation
- Sliding Window — efficient subarray and substring patterns
- System Design — for the later interview rounds